Nano-Sweep Service — bounded swarm scan for DB inconsistencies

SPEC-SW-001 v0.1 DRAFT Jeremie Nunez

1 Context

The knowledge base grows faster than a human can audit it. We need a continuous, cost-bounded process that scans grouped records, flags inconsistencies (missing fields, suspicious values, broken relationships), and hands the output to a human reviewer. This spec defines the *producer* side: a batched swarm of small language-model calls that emits draft findings into a pending buffer. It is never allowed to mutate the source of truth.

2 Goals

- Scan the full entity space in bounded wall-time and bounded token cost per run.
- Partition records into stable batches so each nano call sees a coherent, small context.
- Emit structured findings (category, severity, affected count, optional resolution payload) into the pending buffer.
- Incorporate past reviewer feedback (accepted / rejected) into subsequent prompts to reduce repeat false-positives.
- Surface a machine-readable `SweepResult` to trigger downstream notification.

3 Non-Goals

Non-Goal

This service does *not* write to the source-of-truth database, does not decide whether a finding is correct, and does not notify end users directly. Acceptance, rate-limiting, and dispatch are the responsibility of SPEC-SW-002; application of accepted fixes is the responsibility of SPEC-SW-003.

4 Invariants

Invariant

The sweep service writes only to the pending finding buffer (Redis). It MUST NOT issue `INSERT` / `UPDATE` / `DELETE` against the source-of-truth tables.

Invariant

Every emitted finding has a bounded shape: `category` $\in$ known enum, `severity` $\in$ {`critical`, `warning`, `info`}, `title` $\leq$ 200 chars, `description` $\leq$ 1000 chars. Unknown values are coerced to safe defaults; malformed JSON yields an empty batch result, never a crash.

Invariant

A single sweep run produces a monotonically non-decreasing counter: `totalCalls` $\geq$ `successCalls` ≥ 0, and `suggestionsStored` ≥ 0.

5 Interface

```
export interface SweepResult {
  totalGroups: number;        // partitions scanned
  totalCalls: number;         // nano calls attempted
  successCalls: number;       // calls that returned valid JSON
  suggestionsStored: number; // findings written to pending buffer
  wallTimeMs: number;
  estimatedCost: number;      // USD, computed from token accounting
}

export type SweepCompleteCallback = (r: SweepResult) => Promise<void>;

export class NanoSweepService {
  constructor(entityRepo: EntityRepository, findingRepo:
     FindingRepository);
  onComplete(cb: SweepCompleteCallback): void;
  sweep(maxGroups?: number, mode?: SweepMode): Promise<SweepResult>;
}
```

6 Scenarios

Scenario: Nominal batch scan

Given the entity repository returns N groups with aggregated stats
When `sweep()` is invoked with no cap
Then the service builds $\lceil N/\text{BATCH_SIZE} \rceil$ batches
And dispatches them through the nano wave caller
And every parsed finding is appended to the pending buffer
And onComplete callbacks fire exactly once with the aggregate `SweepResult`.

Scenario: Partial nano failure

Given 10 batches are dispatched
When 3 nano calls return non-parseable text or error out
Then `successCalls = 7` and `totalCalls = 10`
And the 7 successful batches still produce findings
And the run completes without throwing.

Scenario: Feedback-informed prompt

Given the feedback log contains a prior rejection for category `price_anomaly` on group G
When a new batch contains group G
Then the prompt sent to nano includes that prior rejection as a bullet
And the parser coerces any returned category outside the enum to `enrichment`.

Scenario: Cap respected

Given the repository returns 1000 groups
When `sweep(maxGroups=50)` is called
Then exactly 50 groups are scanned
And the remaining 950 are not read, not prompted, not charged.

Scenario: No writes to source-of-truth

Given the sweep runs to completion
When the source-of-truth tables are inspected before and after
Then their row counts and contents are byte-identical.

7 Acceptance Criteria

Acceptance Criterion AC-1

Given N groups and a batch size B, the service issues exactly $\lceil N/B \rceil$ nano calls.

Acceptance Criterion AC-2

When a nano response is not valid JSON, the batch contributes zero findings and the run does not throw.

Acceptance Criterion AC-3

Every finding written to the pending buffer has `category`, `severity`, `title`, `description` satisfying the bounded-shape invariant.

Acceptance Criterion AC-4

No `INSERT` / `UPDATE` / `DELETE` is issued against source-of-truth tables during a sweep (verified by DB hook / spy).

Acceptance Criterion AC-5

`onComplete` callbacks are invoked exactly once per sweep, after the pending buffer write has committed.

Acceptance Criterion AC-6

The `estimatedCost` field is non-negative and reproducible for identical input/output token counts.

Acceptance Criterion AC-7

When `maxGroups` is provided, the repository is queried for at most that many groups and no more nano calls are issued than $\lceil \text{maxGroups}/B \rceil$.

8 Traceability

AC	Test file	Test name
AC-1	`tests/unit/nano-sweep.batching.spec.ts`	`batches groups into fixed size`
AC-2	`tests/unit/nano-sweep.parser.spec.ts`	`malformed JSON yields empty`
AC-3	`tests/unit/nano-sweep.parser.spec.ts`	`enforces bounded shape`
AC-4	`tests/integration/nano-sweep.readonly.spec.ts`	`does not mutate source`
AC-5	`tests/unit/nano-sweep.callbacks.spec.ts`	`onComplete fires once`
AC-6	`tests/unit/nano-sweep.cost.spec.ts`	`cost is deterministic`
AC-7	`tests/unit/nano-sweep.cap.spec.ts`	`maxGroups caps issued calls`

9 Open Questions

- Should batch size adapt to group cardinality, or stay a tuned constant?
- Do we persist raw nano responses for audit, or only parsed findings?
- How is partial-run resumability handled on worker crash?